

Part-1

Run	Hits	π
1	12,126,628	0.970130
2	12,458,276	0.996662
3	13,018,783	1.041503
4	12,778,656	1.022292
5	12,815,865	1.025269

```
Last login: Thu Sep 17 14:42:38 on ttys000
altxw@MacBook-Air-Altynay ~ % ls
Applications  Documents  IdeaSnapshots  Movies  Pictures  benchmark  task3
Desktop       Downloads  Library        Music   Public    benchmark.cpp  task3.cpp
altxw@MacBook-Air-Altynay ~ % cd desktop
altxw@MacBook-Air-Altynay desktop % javac Part1.java
java Part1
Run 1: hits=12126628, pi=0.970130, time=118,221 ms
Run 2: hits=12458276, pi=0.996662, time=133,437 ms
Run 3: hits=13018783, pi=1.041503, time=104,596 ms
Run 4: hits=12778656, pi=1.022292, time=96,986 ms
Run 5: hits=12815865, pi=1.025269, time=97,282 ms
altxw@MacBook-Air-Altynay desktop %
```

The program produced incorrect π values between 0.970130 and 1.041503. The error is caused by a data race on the shared totalHits variable. Multiple threads perform totalHits++ concurrently, so some updates are overwritten and lost.

Part-2

Version	Threads	π	Runtime
Single-threaded	1	3.141500	173.980 ms
Synchronized	4	3.141827	1683.910 ms

```
altxw@MacBook-Air-Altynay desktop % javac Part2.java
java Part2
Single-thread: pi=3,141500
Single-thread time: 173,980 ms
Synchronized 4-thread: pi=3,141827, time=1683,910 ms
altxw@MacBook-Air-Altynay desktop %
```

Part-3

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T_1/T_n)	Efficiency (Speedup/T)
1	1228.573	1.000X	100.00%
2	1160.168	1.059X	52.95%

4	1341.610	0.916X	22.89%
8	1673.455	0.734X	9.18%
16	1696.823	0.724X	4.53%
32	1692.326	0.726X	2.27%

```

altxw@MacBook-Air-Altnay desktop % javac Part3.java
[java Part3
Threads=1 | Runtime=748,700 ms | Pi=3,141736
Starting benchmark...
Threads=1 | Runtime=787,013 ms | Pi=3,141747
Speedup=1,000X | Efficiency=100,00%

Threads=2 | Runtime=344,139 ms | Pi=3,141631
Speedup=2,287X | Efficiency=114,35%

Threads=4 | Runtime=179,184 ms | Pi=3,141666
Speedup=4,392X | Efficiency=109,81%

Threads=8 | Runtime=127,643 ms | Pi=3,141469
Speedup=6,166X | Efficiency=77,07%

Threads=16 | Runtime=122,900 ms | Pi=3,141735
Speedup=6,404X | Efficiency=40,02%

Threads=32 | Runtime=123,901 ms | Pi=3,141616
Speedup=6,352X | Efficiency=19,85%

altxw@MacBook-Air-Altnay desktop % █

```

QA

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

Answer: An 8-core CPU cannot run 16 threads at the same time on 16 separate physical cores. With 16 threads, multiple threads must share the available CPU cores and resources. This creates additional scheduling and resource overhead, so 16 threads do not run twice as fast as 8 threads.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Answer: The synchronized version was slower because every increment of the shared counter requires synchronization. Only one thread can update the shared variable at a time, while the other threads have to wait. The synchronization overhead and contention can therefore be greater than the performance benefit of using multiple threads.